2025/06/17/AI_R&D_Center

動的計画法を用いた編集距離の計算

23B1014　旦保惟果

やりたいことを、できることに。
湘南工科大学
SHONAN INSTITUTE OF TECHNOLOGY
AI R&D CENTER

目次

- 文字列の類似性をどう表すか
- 編集距離とは
- 動的計画法とは
- DPによる編集距離の計算
- DPテーブルの遷移
- まとめ

この二つの文字列はどのくらい似ている？

kitten

sitting

文字列の類似性をどう表すか？

- 人間は「なんとなく似ている」と感覚的に判断できる

- しかしコンピュータには「なんとなく似ている」が判断できない
  - 定量的に「似ているか」を判断する必要がある
  - 例：スペルチェック, DNA配列解析

- 2つの文字列の類似性を定量的に評価するために用いられるのが今回話す**編集距離**

応用例

- スペルチェック
  - 入力された単語と辞書内の単語の編集距離を計算して誤字脱字を直す
- DNAの類似度
  - 異なるDNA配列の類似度を編集距離で評価して生物学的関係を分析する
- 自然言語処理
  - 単語や文章の類似性判定（剽窃していないか），音声認識の誤り訂正など
- 検索エンジンのサジェスト機能
  - ユーザーの入力ミスを検出し正しい検索ワードを予測する

編集距離(Levenshtein distance)とは

- 2つの文字列SとTに対して，編集距離を以下のように定義する.
- $d(S,T) :=$ 文字列Sを文字列Tに変換するために必要な最小の編集操作回数

- 次の3つの操作ができる
  - 挿入：文字列の任意の場所に好きな文字を1文字追加する
    - ex) “Cat” → “Cart”(rを挿入)
  - 削除：文字列の任意の場所から一文字を消去する
    - ex) “Cat” → “Ct”(aを削除)
  - 置換：文字列の中の一文字を別の好きな文字に置き換える
    - ex) “Cat” → “Bat”(CをBに置換)

編集距離(Levenshtein distance)とは

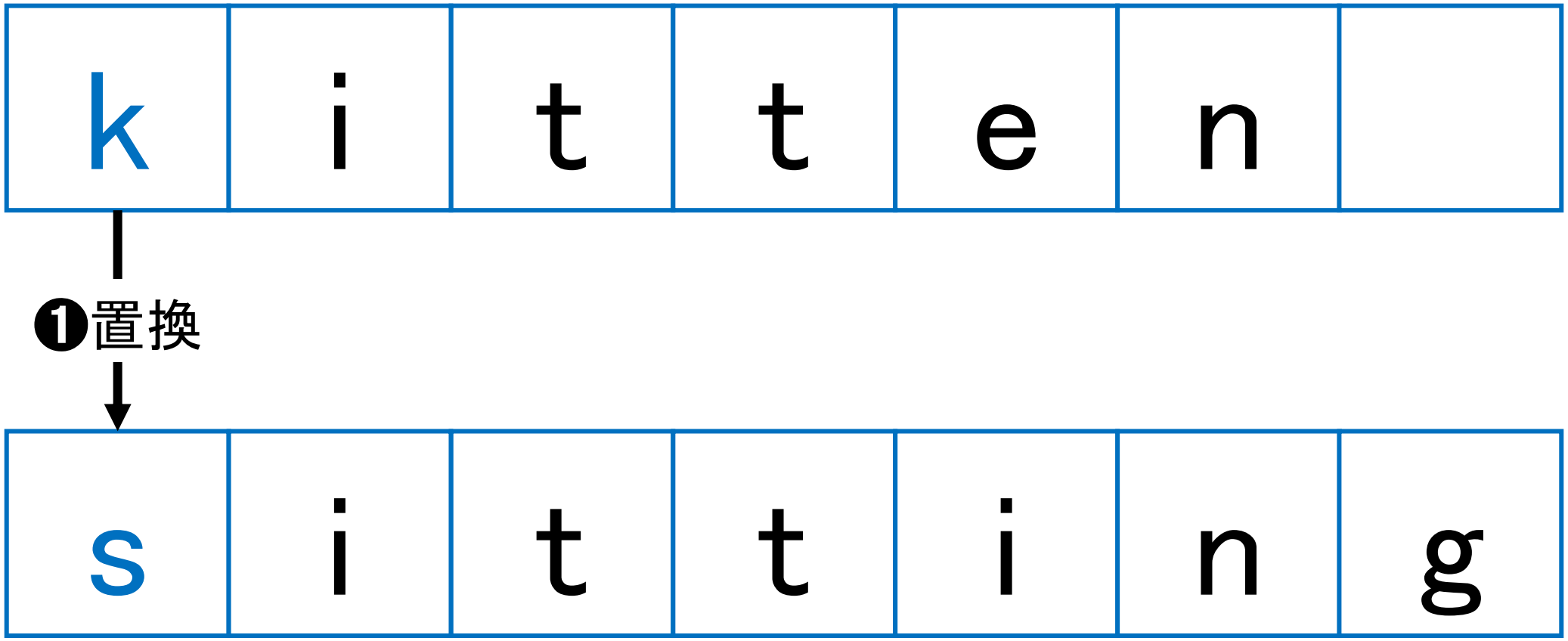
- 例)S=“kitten”,T=“sitting”

k	i	t	t	e	n	

s	i	t	t	i	n	g

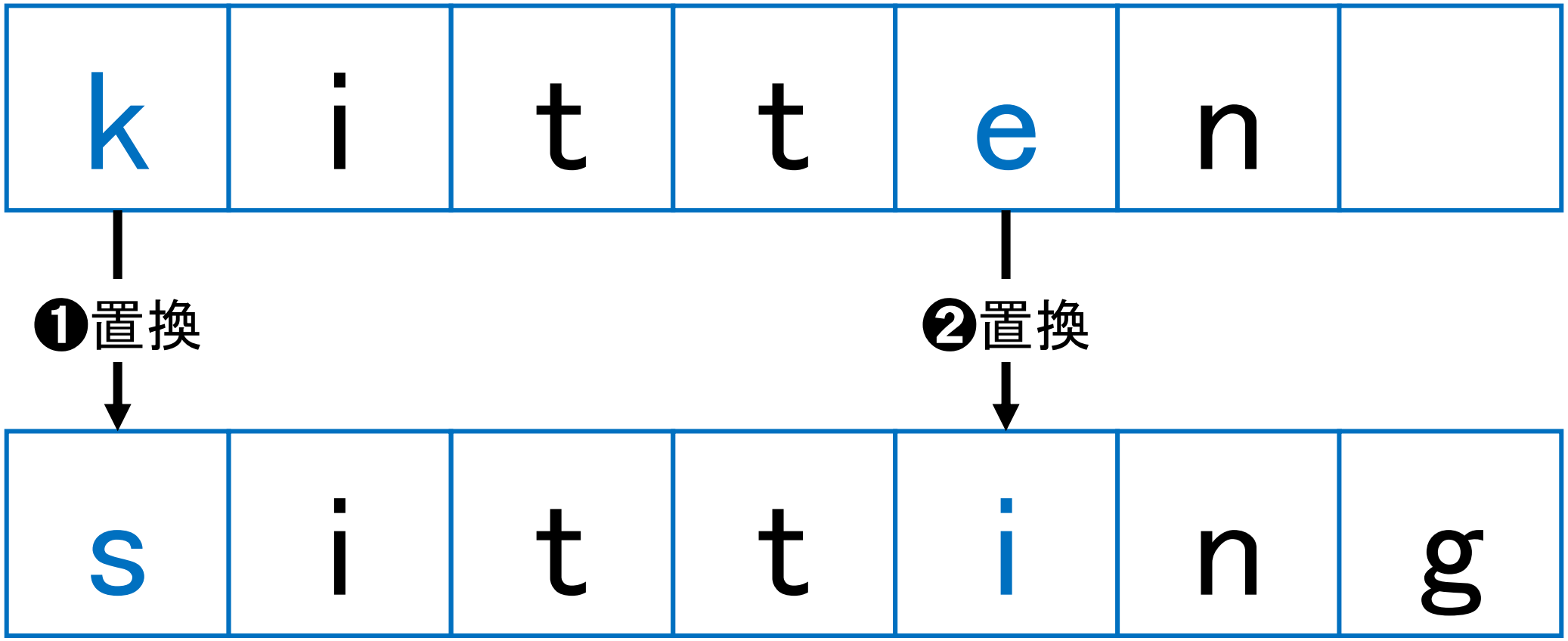
編集距離(Levenshtein distance)とは

- 例)S=“kitten”,T=“sitting”



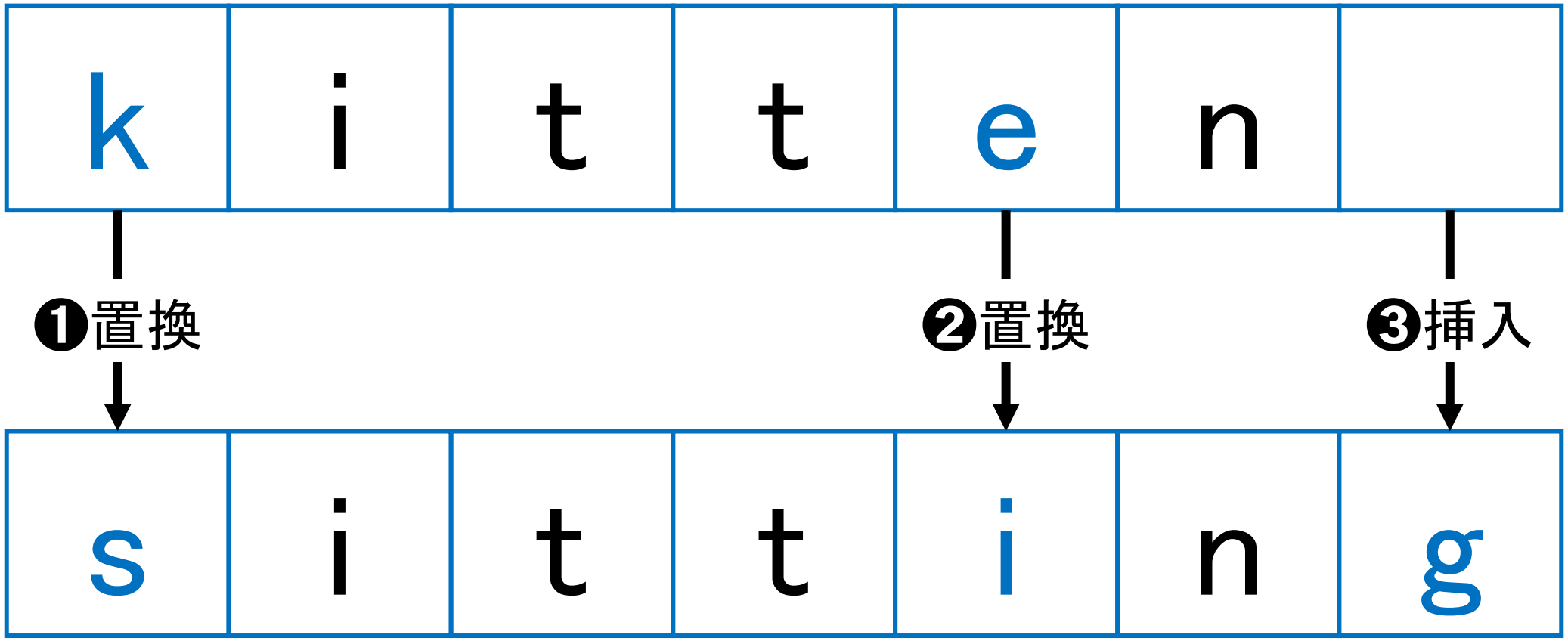
編集距離(Levenshtein distance)とは

- 例)s=“kitten”,t=“sitting”



編集距離(Levenshtein distance)とは

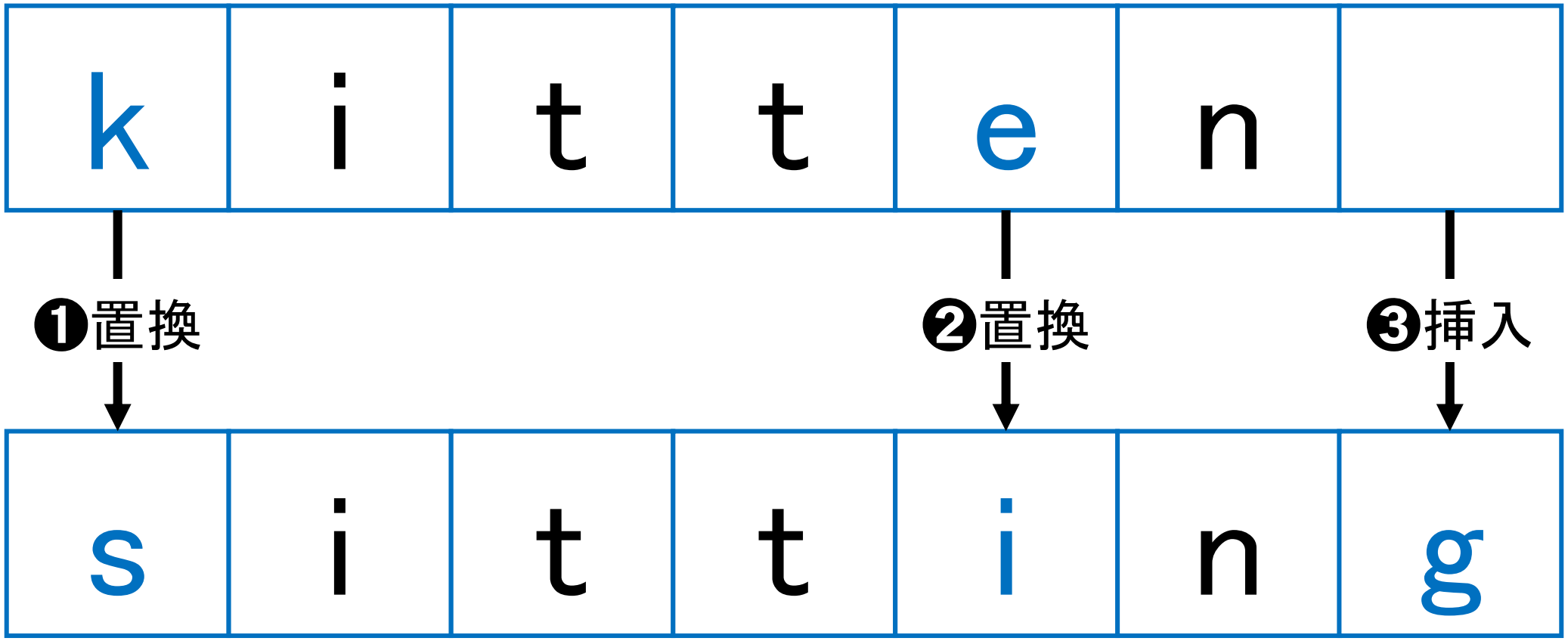
- 例)S=“kitten”,T=“sitting”



編集距離(Levenshtein distance)とは

- 例)S="kitten",T="sitting"

編集距離:3



考え方

- 一文字づつ挿入・削除・置換のすべてのパターンを考えると, 計算量が爆発的に増加してしまう

- そこで動的計画法を用いることで, 計算を効率的に行う

動的計画法とは

- **動的計画法**(Dynamic Programming, 以下DP)は, 大きな問題を小さな部分問題に分割して解く手法

- 一度計算した答えを記憶しておくことで, 何度も同じ計算をしなくて済む
⇒計算量を削減できる

- 例
  - グラフの最短経路問題
  - 日本語形態素解析
  - 部分和問題, ナップサック問題
  - フィボナッチ数列

DPによる編集距離の計算（DP定義と初期化）

- DPの定義
  - $dp[i][j]$:=文字列Sの0~i文字目までと文字列Tの0~j文字目までを一致させるための最小の操作回数
  - 文字列の0文字目を空文字列(””)と考える

- 最後にどんな操作をしたかを注目して考える
  - 一つ前の状態から最後にどんな操作をしたか

- 初期化
  - $dp[0][0] = 0$(空文字列から空文字列への距離)
  - $dp[i][0] = i$ （文字列Sのi文字目までを空文字列にするには、i回の削除が必要）
  - $dp[0][j] = j$ (空文字列を文字列Tのj文字目までにするには、j回の挿入が必要)

DPによる編集距離の計算（更新式）

- Sのi文字目とTのj文字目が一致しているとき：
  - $dp[i][j] = dp[i-1][j-1]$
  - すでにSの$i-1$文字目までとTの$j-1$文字目までを一致させているならi文字目とj文字目が同じなので，操作は不要
    ⇒よって，コストは増えない

0	1	2	$\cdots$	$i-2$	$i-1$	i

$\neq$

0	1	2	$\cdots$	$j-2$	$j-1$	j

DPによる編集距離の計算（更新式）

- Sのi文字目とTのj文字目が一致していないとき：
  - 最後の操作が置換：
    - $dp[i][j] = dp[i-1][j-1] + 1$
    - すでにSの$i-1$文字目までとTの$j-1$文字目までを一致させているならSのi文字目をTのj文字目に置き換えれば一致する

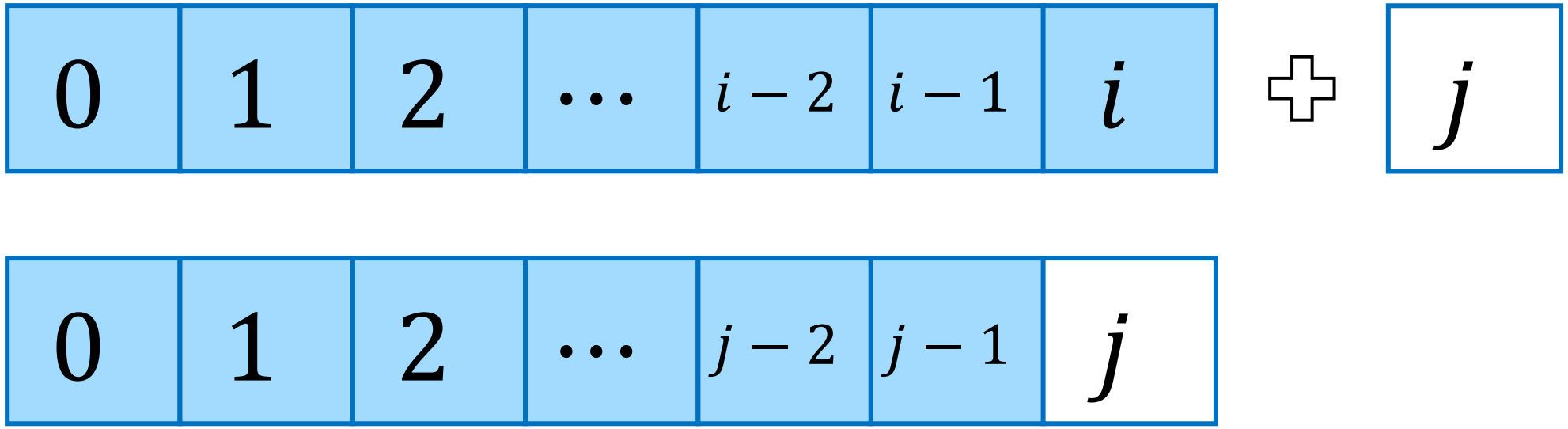
0	1	2	$\cdots$	$i-2$	$i-1$	i

$\neq$

0	1	2	$\cdots$	$j-2$	$j-1$	j

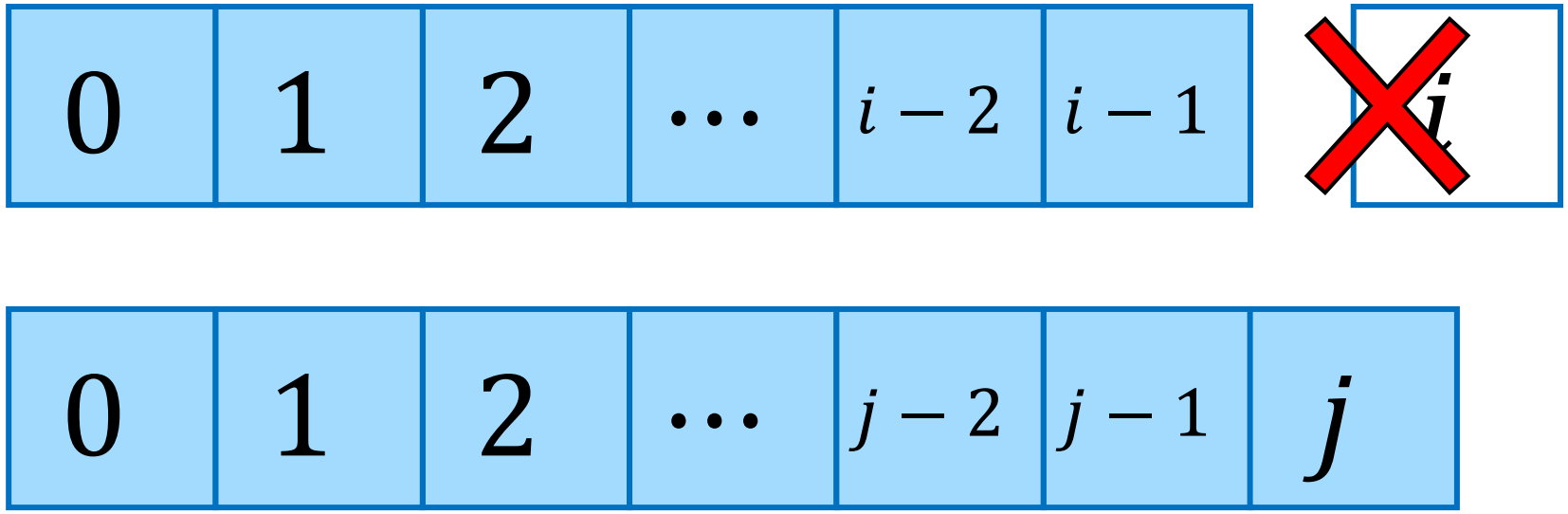
DPによる編集距離の計算（更新式）

- Sのi文字目とTのj文字目が一致していないとき：
  - 最後の操作が挿入：
    - $dp[i][j] = dp[i][j-1] + 1$
    - すでにSのi文字目までとTの$j-1$文字目までを一致させているなら文字列SにTのj文字目を挿入すれば一致する



DPによる編集距離の計算（更新式）

- Sのi文字目とTのj文字目が一致していないとき：
  - 最後の操作が削除：
    - $dp[i][j] = dp[i-1][j] + 1$
    - すでにSの$i-1$文字目までとTのj文字目までを一致させているなら文字列Sのi文字目を消去すれば一致する



DPによる編集距離の計算（更新式）

- 答えは最小操作回数なので, 更新式の中から最小値をとる

- $$dp[i][j] = min\begin{Bmatrix} dp[i-1][j-1](S[i] = T[j]), \\ dp[i-1][j-1] + 1, \\ dp[i][j-1] + 1, \\ dp[i-1][j] + 1 \end{Bmatrix}.$$

DPテーブルの遷移

$i \setminus j$	0(“”)	1(s)	2(si)	3(sit)	4(sitt)	5(sitti)	6 (sittin)	7 (sitting)
0(“”)	0	1	2	3	4	5	6	7
1(k)	1							
2(ki)	2							
3(kit)	3							
4(kitt)	4							
5(kitte)	5							
6(kitten)	6							

$dp[1][1]$の更新
$S[:1]$ =“k”を$T[:1]$ =“s”にするための操作の最小回数を考える

DPテーブルの遷移

$i \setminus j$	0(“”)	1(s)	2(si)	3(sit)	4(sitt)	5(sitti)	6 (sittin)	7 (sitting)
0(“”)	0	1	2	3	4	5	6	7
1(k)	1							
2(ki)	2							
3(kit)	3							
4(kitt)	4							
5(kitte)	5							
6(kitten)	6							

$dp[1][1]$の更新

$S[:1]$ =“k”を$T[:1]$ =“s”にするための操作の最小回数を考える

$S[1] \neq T[1]$なので

$$min\begin{Bmatrix} dp[0][0]+1=0+1(\text{置換}), \\ dp[1][0]+1=1+1(\text{挿入}), \\ dp[0][1]+1=1+1(\text{削除}) \end{Bmatrix}$$

よって$dp[1][1]=1$

DPテーブルの遷移

$i \setminus j$	0(“”)	1(s)	2(si)	3(sit)	4(sitt)	5(sitti)	6 (sittin)	7 (sitting)
0(“”)	0	1	2	3	4	5	6	7
1(k)	1	1						
2(ki)	2							
3(kit)	3							
4(kitt)	4							
5(kitte)	5							
6(kitten)	6							

$dp[1][1]$の更新

$S[:1]$ =“k”を$T[:1]$ =“s”にするための操作の最小回数を考える

$S[1] \neq T[1]$なので

$$min\begin{Bmatrix} dp[0][0]+1=0+1(\text{置換}), \\ dp[1][0]+1=1+1(\text{挿入}), \\ dp[0][1]+1=1+1(\text{削除}) \end{Bmatrix}$$

よって$dp[1][1]=1$

DPテーブルの遷移

$i \setminus j$	0(“”)	1(s)	2(si)	3(sit)	4(sitt)	5(sitti)	6 (sittin)	7 (sitting)
0(“”)	0	1	2	3	4	5	6	7
1(k)	1	1	2	3	4	5	6	7
2(ki)	2	2						
3(kit)	3							
4(kitt)	4							
5(kitte)	5							
6(kitten)	6							

$dp[2][2]$の更新
$S[:1]$ =“ki”を$T[:1]$ =“si”にするための操作の最小回数を考える

DPテーブルの遷移

$i \setminus j$	0(“”)	1(s)	2(si)	3(sit)	4(sitt)	5(sitti)	6 (sittin)	7 (sitting)
0(“”)	0	1	2	3	4	5	6	7
1(k)	1	1	2	3	4	5	6	7
2(ki)	2	2						
3(kit)	3							
4(kitt)	4							
5(kitte)	5							
6(kitten)	6							

$dp[2][2]$の更新
$S[:2]$ =“ki”を$T[:2]$ =“si”にするための操作の最小回数を考える
$S[2] = T[2]$なので
$dp[2][2] = dp[1][1]$

DPテーブルの遷移

$i \setminus j$	0(“”)	1(s)	2(si)	3(sit)	4(sitt)	5(sitti)	6 (sittin)	7 (sitting)
0(“”)	0	1	2	3	4	5	6	7
1(k)	1	1	2	3	4	5	6	7
2(ki)	2	2	1					
3(kit)	3							
4(kitt)	4							
5(kitte)	5							
6(kitten)	6							

$dp[2][2]$の更新
$S[:2]$ =“ki”を$T[:2]$ =“si”にするための操作の最小回数を考える
$S[2] = T[2]$なので
$dp[2][2] = dp[1][1]$

DPテーブルの遷移

$i \setminus j$	0(“”)	1(s)	2(si)	3(sit)	4(sitt)	5(sitti)	6 (sittin)	7 (sitting)
0(“”)	0	1	2	3	4	5	6	7
1(k)	1	1	2	3	4	5	6	7
2(ki)	2	2	1	2	3	4	5	6
3(kit)	3	3	2	1	2	3	4	5
4(kitt)	4	4	3	2	1	2	3	4
5(kitte)	5	5	4	3	2	2	3	4
6(kitten)	6	6	5	4	3	3	2	**3**

よって，編集距離は3

まとめ

- 編集距離は、2つの文字列の違いを最小限の編集操作（挿入・削除・置換）で埋めるための手法

- 編集距離を用いると、文字列の類似性を定量的に評価し、さまざまな文字列処理の問題を効率的に解くことができる